

Note-ID Resolution Review and Fix

Summary of a review of `resolve_note_ids_for_highlight` and the melodic n-gram note-ID pipeline in `src/encoding_music_mcp/tools/intervals.py`, prompted by a report that highlighted notation only worked for some voice parts (staves) in a score.

The symptom

`get_melodic_ngrams` worked correctly, but requests to highlight the resulting note IDs in notation only resolved for *some* voices in a piece, not others.

Root cause

`resolve_note_ids_for_highlight` (via `_resolve_note_id_spans`) looked up each requested highlight span by `(measure, beat, offset)` in a lookup table built by `_build_part_note_events`. That function parsed the raw MEI XML directly and computed each voice's position by **summing the `dur.ppq` of whatever `<note>/<rest>/<mRest>` elements it found**, per part, per measure — with no cross-check against the measure's declared length. The spans being resolved, however, came from CRIM Intervals' own `piece.detailIndex()` coordinates (via `music21`) — a second, independently computed timing system.

The two systems were assumed to always agree. They didn't.

Reproduction

Using `CRIM_Mass_0031_2.mei` (a 4/2-meter Renaissance mass movement):

- Staff 1 (“Superius”) had tacet rests in measures 1, 2, 22, and 51 encoded as **half the measure's actual duration** (1024 ppq instead of 2048 ppq) — a common mensural-notation convention: a single rest symbol standing in for “not yet entered,” rather than a metrically exact duration.
- `_build_part_note_events` never padded a measure to its declared length, so each of these events left that voice's running offset **4 quarters short**. Over 4 occurrences, that's a 16-quarter (2-measure) drift.
- By measure 90, the code labeled a note onset “measure 90,” while CRIM's own timeline said the same physical offset was “measure 88” — an exact 2-measure gap.

From that point forward, none of that voice's (`measure`, `beat`, `offset`) keys matched CRIM's coordinates, so `note_ids` came back empty for every subsequent highlight request on that voice.

Measured impact on `CRIM_Mass_0031_2.mei` (`get_melodic_ngram_matches`, `n=4`), before the fix:

```
column 1: 44/167 occurrences empty (26%)
column 2: 23/217 occurrences empty (11%)
column 3: 60/176 occurrences empty (34%)
column 4: 50/165 occurrences empty (30%)
```

Different voices drifted by different amounts depending on how many short-rest measures preceded the query — hence “only some voice parts” resolved.

The fix

Instead of reconstructing an independent timeline from raw XML ticks, pull note timing directly from CRIM's own `music21` objects — the same objects that produce the (`measure`, `beat`, `offset`) coordinates used everywhere else in the codebase.

Removed (`_get_staff_ppq`, `_iter_layer_events`, the old `_build_part_note_events(filepath)`, and the now-unused ET import / `_MEI_NS` / `_XML_ID` constants):

```
def _get_staff_ppq(root: ET.Element) -> dict[str, int]: ...
def _iter_layer_events(element, current_ppq) -> list[dict]: ...
def _build_part_note_events(filepath: Path) -> dict[str, list[dict]]:
    """Parse MEI and return sounded-note events for each CRIM part number."""
    root = ET.parse(filepath).getroot()
    staff_ppq = _get_staff_ppq(root)
    ...
    for measure in root.findall("./mei:measure", _MEI_NS):
        ...
        for staff in measure.findall("mei:staff", _MEI_NS):
            ...
            for event in layer_events:
                dur_ppq = event["dur_ppq"]
                if event["kind"] == "note" and event["note_ids"]:
                    onset_q = global_offsets_ppq[part_label] / current_ppq
                    beat = 1 + (measure_offset_ppq / current_ppq)
                    part_events[part_label].append({...})
                measure_offset_ppq += dur_ppq
                global_offsets_ppq[part_label] += dur_ppq # <- drifts silently
```

Added:

```
def _note_ids_from_m21_obj(obj: Any) -> list[str]:
    """Return MEI xml:id values carried by a music21 note/chord object.

    music21's MEI importer sets `.id` to the real xml:id string when one is
    present; an object with no xml:id keeps music21's default int id, so
    filtering on `str` reliably distinguishes real ids from that default.
    """
    if obj is None or getattr(obj, "isRest", False):
        return []
    if getattr(obj, "isChord", False):
        return [n.id for n in obj.notes if isinstance(n.id, str)]
    return [obj.id] if isinstance(obj.id, str) else []

def _build_part_note_events(piece: Any) -> dict[str, list[dict[str, Any]]]:
    """Return sounded-note events for each CRIM part number.

    Timed via the same music21 offset/measure/beat machinery CRIM Intervals
    uses for its own analysis dataframes (numberParts + detailIndex over
    `_getM21ObjsNoTies()`), so results share one coordinate system with the
    melodic n-gram output instead of a second, independently-computed one.
    """
    objs = piece.numberParts(piece._getM21ObjsNoTies())
    detailed = piece.detailIndex(objs, offset=True)

    part_events: dict[str, list[dict[str, Any]]] = {}
    for part_label in detailed.columns:
        events: list[dict[str, Any]] = []
        for (measure, beat, offset), obj in detailed[part_label].dropna().items():
            note_ids = _note_ids_from_m21_obj(obj)
            if not note_ids:
                continue
            events.append(
                {
                    "measure": float(measure),
                    "beat": float(beat),
                    "offset": float(offset),
                    "duration": float(obj.quarterLength),
                    "note_ids": note_ids,
                }
            )
```

```
part_events[str(part_label)] = events
return part_events
```

`_resolve_note_id_spans`, `_build_note_id_matches`, and `resolve_note_ids_for_highlight` now take a loaded piece (from `importScore`) instead of a bare filepath; the span-matching logic itself (`note_count`, duration spans, part-label filtering) was untouched.

Verification

Automated tests

```
uv run pytest tests/test_intervals.py -v
```

All 30 existing tests pass, including the pre-existing `resolve_note_ids_for_highlight` coverage.

Before/after on real files

```
CRIM_Mass_0031_2.mei total 725 empty 0    (was 11-34% empty per voice)
CRIM_Model_0003.mei  total 176 empty 0
CRIM_Mass_0013_5.mei total 856 empty 0
CRIM_Mass_0028_3.mei total 2248 empty 0
```

The specific failing case identified during root-causing (`CRIM_Mass_0031_2.mei`, part "1", measure 90, beat 4.5, offset 719.0) now resolves correctly:

```
{
  "filename": "CRIM_Mass_0031_2.mei",
  "spans": [
    {
      "column": "1",
      "start_measure": 90.0,
      "start_beat": 4.5,
      "start_offset": 719.0,
      "note_count": 1,
      "index": 0,
      "matched_parts": ["1"],
      "note_ids": ["m-3670"]
    }
  ]
}
```

```
}  
]  
}
```

Full-file coverage check (Bach_BWV_0772.mei)

To check literal 1:1 coverage against every `<note>` element in the raw MEI, not just sampled queries: 466 `<note>` elements with an `xml:id` in the file; 463/466 (99.4%) resolve. The 3 gaps were traced to a **separate, pre-existing bug in `crim_intervals` itself** (not introduced by this fix, and not specific to note-ID resolution):

- `crim_intervals`'s `_getPartSeries()` collapses a chord to its highest note via `max(chord.notes)`. Extracting a `Note` from `Chord.notes` detaches it from its stream context — its `.activeSite` becomes `None`, so `.offset` resolves to `0.0` instead of the chord's real position. This silently drops the chord's other notes from *every* CRIM-based tool (`get_notes`, `get_melodic_intervals`, `n-grams`), not just highlighting. Confirmed directly: `piece.notes()` shows `NaN` for that voice/position independent of any code in this repo.

(An initial hypothesis that 7 other “missing” notes were a beam-related issue was wrong — they turned out to be tied continuation notes, correctly excluded by `_getM210bjsNoTies()` since a tie represents one sounding note, not two attacks. Verified via the standalone `<tie startid= endid=>` MEI elements pointing at them, e.g. `<tie endid="#n112w31q" startid="#n5cre6m"/>.`)

End-to-end pattern test (Bach_BWV_0772.mei)

Used the recommended shortcut (`get_melodic_ngram_matches`) to find the 3 most common melodic 4-grams and confirm every occurrence resolves in both voices:

```
-2_3_-2_3:    27 occurrences, voices {'1': 14, '2': 13}, 0 empty  
3_-2_3_-2:    27 occurrences, voices {'1': 14, '2': 13}, 0 empty  
-2_-2_3_-2:   21 occurrences, voices {'1': 12, '2': 9},  0 empty
```

All resulting note IDs (111 / 111 / 104 unique notes respectively) were also confirmed to exist as real, addressable `id="..."` elements in the rendered notation SVG output across every page, e.g.:

```
{
  "filename": "Bach_BWV_0772.mei",
  "highlight_note_ids": ["n10asc35", "n11c5xy4", "n15e0j0z", ...],
  "start_measure": 9,
  "end_measure": 12
}
```

passed to `show_notation_highlight` and visually confirmed in the MCP Inspector's **Apps** tab.

The note-ID → highlight pipeline (for reference)

1. `get_melodic_ngrams` returns a pattern table (Measure, Beat, Offset rows × voice columns) — no note IDs by default.
2. To get note IDs, either:
 - Re-call with `include_note_ids=True`, or
 - Use `get_melodic_ngram_matches`, which resolves and groups occurrences by pattern string in one call (the recommended approach), or
 - Manually build a span from a pattern-table row and call `resolve_note_ids_for_highlight`:

```
resolve_note_ids_for_highlight(
    "Bach_BWV_0772.mei",
    [{"column": "1", "start_measure": 1.0, "start_beat": 1.25,
      "start_offset": 0.25, "note_count": 5}], # note_count = n + 1
)
```

3. Pass the resulting `note_ids` to `show_notation_highlight` to render.

Known remaining limitation

Chords collapsed via `crim_intervals`'s `_getPartSeries()` lose their non-highest notes from all analysis, including note-ID resolution. This is an upstream library issue, not fixed here. It could be worked around in `_build_part_note_events` by detecting chords from the raw `note/rest` objects before `CRIM`'s `max()` collapse discards their context, if needed.

Local testing options

- **MCP Inspector** (browser, reflects this local checkout): `uv run fastmcp dev inspector dev_server.py:mcp` — **Tools** tab for raw JSON, **Apps** tab for rendered/highlighted notation.

- **Claude Desktop:** point `claude_desktop_config.json`'s `mcpServers` entry at this local directory instead of the published package:

```
{
  "mcpServers": {
    "encoding-music-mcp": {
      "command": "uv",
      "args": ["--directory", "/absolute/path/to/encoding-music-mcp", "run", "encoding-m"]
    }
  }
}
```

then fully quit and restart Desktop.

- **Automated tests:** `uv run pytest tests/test_intervals.py -v`